Machine Learning in Engineering Science
Week 10: Reproducible ML Workflow Studio

NCKU + LAiMM / Prof. Chi-Hua Yu

Fall 2026 / Monday, 9 November 2026

Contents

1 Week 10: Reproducible ML Workflow Studio **2**

2 Stable identity, not a seed alone **2**

3 The packaged regression fixture and its limits **2**

4 Baseline, selection and held-out evidence **3**

5 A checkable function explanation **5**

6 What belongs in the handover **5**

7 Replay outside notebook memory **6**

8 Test deliberate failures on copies **6**

9 Eight core exercises and two optional challenges **7**

10 Instructor execution and limits **8**

11 Summary and examination bridge **8**

12 Primary-source reading **8**

1 Week 10: Reproducible ML Workflow Studio

Monday, 9 November 2026, 14:00–17:00. The class contains 60 minutes of explanation and 120 minutes of practice. H1 uses 45/15 minutes, H2 uses 15/45, and H3 is a 60-minute studio. Eight core exercises and two optional challenges are formative in-class practice, not new weekly graded homework. The next scheduled meeting is the written and oral midterm examination; this studio does not replace either examination.

The goal is a portable handover: another interpreter receives frozen files, checks their contract, reconstructs a declared model, and compares predictions by stable row identity. A matching screenshot, aggregate score or second notebook variable is not sufficient. Week 9 studied where a model fails; Week 10 asks whether the evidence supporting that analysis can travel.

1.1 Computational replay has a bounded meaning

Here, replay means recomputing predictions from the same frozen input records and declared procedure. A new interpreter and working directory remove the notebook's global variables and fitted-object memory. They do not create a new dataset, machine, package environment or physical experiment. Thus numerical agreement is narrower than independent scientific replication. A reproducible model can still be inaccurate or inapplicable.

A two-row counterexample clarifies the distinction between score and prediction identity. Errors $[0, 2]$ and $[2, 0]$ both have MAE 1, but fail on different observations. Compare aligned prediction vectors, not only their mean loss.

2 Stable identity, not a seed alone

A seeded permutation selects positions in an ordered array. If the array order changes first, the same positions may identify different observations. Assign source IDs before shuffling and persist them with their measured values. To reconstruct a cohort, build an ID-keyed lookup and iterate over the saved ID sequence. A set comparison verifies membership; a sequence comparison additionally verifies alignment.

The partition contract requires unique row IDs, no duplicates within any split, pairwise disjoint train/validation/test sets, the expected counts, and a union equal to the full frozen dataset. Disjointness alone does not catch omitted rows. Source row IDs are version-specific dataset identifiers, not globally stable patient identities or predictive features.

3 The packaged regression fixture and its limits

The original Week 10 material used scikit-learn's bundled diabetes regression dataset. We retain its 442 observations and 10 numeric inputs but call `load_diabetes(scaled=False)`. The default loader returns globally centered/scaled inputs; raw loading makes the boundary of training-only fitted preprocessing explicit. We do not assign medical interpretations to individual features.

Use boundary: this is a statistical workflow exercise, not diagnosis, treatment guidance, causal discovery or external clinical validation. A random-row teaching split cannot establish future-time,

new-site or clinical deployment validity. Dataset documentation also cautions that some feature meanings are not fully explicit.

One seeded permutation reserves 265 training, 88 validation and 89 final-test rows. Store exact floats, feature names, row IDs and split lists. The notebook checks finite numeric values, shape, counts and ID membership before fitting.

3.1 An executable interface

The CSV header must exactly match `row_id`, the ten declared feature names in canonical order, and `target`. Duplicate IDs, non-finite values and reordered columns are rejected. Strict rejection is appropriate for a positional model interface; a separately declared and tested name adapter may create a valid canonical interface. Silently guessing feature order is not an acceptable repair.

These checks establish well-formedness relative to a contract, not scientific validity. A finite label can still be wrong, a unique ID can be attached to a wrong observation, and a correctly shaped dataset can be biased.

4 Baseline, selection and held-out evidence

Declare four candidates before final evaluation: a training-target median baseline and training-scaled Ridge models with $\alpha \in \{1, 20, 100\}$. A median minimizes $\sum_i |y_i - c|$ over constant predictions c. For an even sample size, any value between the two middle order statistics is a minimizer. That training property does not guarantee the best held-out score.

For the standardized design matrix Z, scikit-learn Ridge uses a residual-sum-of-squares objective plus an L2 penalty:

$$\min_{b,\beta} \sum_{i \in \text{train}} (y_i - b - z_i^\top \beta)^2 + \alpha \sum_j \beta_j^2.$$

The intercept is not penalized in this formulation. Standardization and all coefficients are fitted on training data only. Stronger regularization changes the fitted function and coefficient norm; validation MAE need not improve monotonically. The training objective and selection metric need not be identical.

Choose the lowest validation MAE, breaking exact ties by the declared order: median, Ridge 1, Ridge 20, Ridge 100. Freeze the selected candidate before opening final-test results. The replay program refits this already selected candidate on training rows; it does not redo selection or refit on training plus validation.

Report the selected candidate and the predeclared median baseline on the same 89 final-test observations. The residual convention is prediction minus observation; positive means overprediction. Inspect one actual large-residual source ID and its observed/predicted values. Do not infer a clinical mechanism from that row or tune alpha after inspecting it.

Candidate	Test n	MAE	RMSE
median	89	65.280899	79.418957
ridge-20	89	48.325756	59.411283

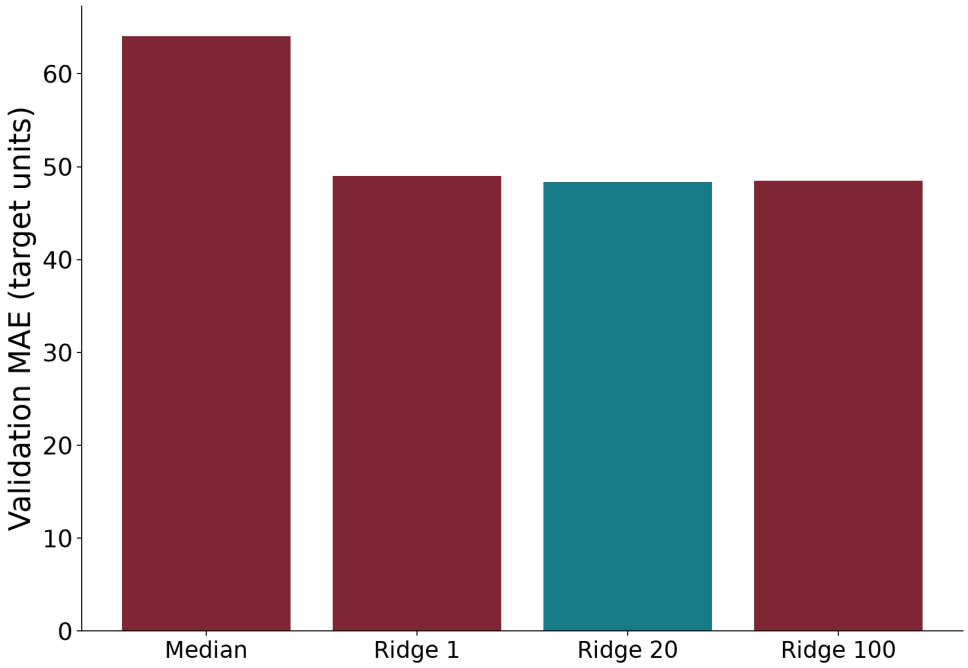


Figure 1: Actual validation comparison on 88 rows. The reference run selects Ridge 20. Students retain their own actual environment and outputs.

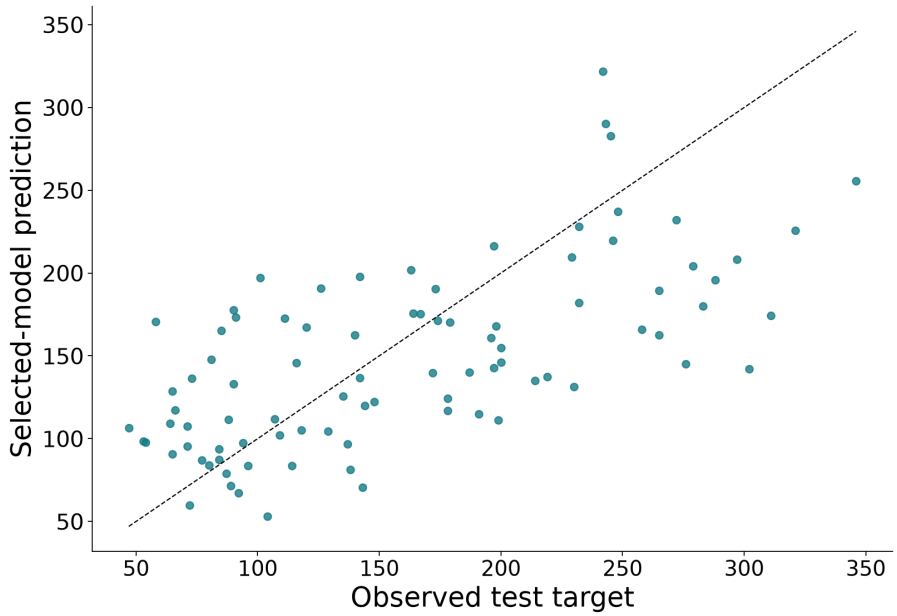


Figure 2: Actual selected-model final-test predictions. A later perfect computational replay does not remove these prediction errors against outcomes.

5 A checkable function explanation

For $z_j = (x_j - \mu_j)/s_j$, standardized coefficient β_j gives

$$\beta_j^{\text{raw}} = \frac{\beta_j}{s_j}, \qquad b^{\text{raw}} = b - \sum_j \frac{\beta_j \mu_j}{s_j}, \qquad f(x) = b^{\text{raw}} + \sum_j \beta_j^{\text{raw}} x_j.$$

Reconstruct predictions and compare them to the pipeline. Increasing one input by $0.1 s_j$ changes the fitted prediction by $0.1\beta_j$, with other coordinates fixed. This is an exact property of the additive fitted function, up to numerical tolerance. It need not correspond to a feasible causal or clinical intervention, especially with correlated features.

If the selected candidate were the constant median, this feature interpretation would not apply. The notebook then inspects the separately predeclared Ridge-20 candidate and records its name. Do not silently interpret a different candidate as the selected one.

6 What belongs in the handover

The payload has six required files:

File	Role
`frozen.csv`	Exact source IDs, features and target
`split.json`	Ordered training, validation and test IDs
`config.json`	Feature order, choices, selected candidate, versions
`replay.py`	Readable validation and fitting procedure
`reference-predictions.csv`	ID-aligned reference outcomes/predictions
`result.json`	Validation/test evidence and limitations

A separate `manifest.json` records the SHA-256 of each payload. It does not hash itself, which would be circular. The exported ZIP also retains actual plots and the fresh-process result.

6.1 Integrity is not authentication

SHA-256 detects changed bytes relative to a known reference digest. Parsing records checks a different identity. Appending a blank CSV line can preserve parsed rows while changing its hash. An unsigned manifest does not authenticate an unknown sender: someone able to replace both payload and digest can create another internally consistent package. Neither hashes nor schemas validate scientific labels or a performance claim.

The lab avoids loading a model pickle by reconstructing from CSV, JSON and readable source. Pickle-based loaders may execute arbitrary code. Executing Python source is also consequential: review and trust the generated runner before running it. A checksum does not make arbitrary downloaded code safe.

7 Replay outside notebook memory

The notebook launches a new Python interpreter in a new temporary working directory, passing only an explicit handover path and output path. The runner first verifies the required-file set and hashes, then validates schema and split membership, then refits the frozen candidate and predicts the saved test IDs. Notebook globals cannot rescue missing files.

Check exact test-ID sequence before numerical comparison. For each value, the comparison uses

$$|p_i^{\text{replay}} - p_i^{\text{reference}}| \le a + r|p_i^{\text{reference}}|, \qquad a = 10^{-8},\ r = 10^{-10}.$$

For reference value 10, the allowed difference is 1.1×10^{-8}. This tolerance is a numerical-agreement rule, not a model-accuracy score. Inspect input bytes, IDs, feature order, configuration and versions before changing it.

The actual local reference maximum prediction difference is 0.0. That establishes a replay under the recorded installed environment, not all future software versions. NumPy's random-stream compatibility documentation imposes strict conditions involving seed, generator, call sequence, build, environment and machine; do not overstate a seed-only guarantee.

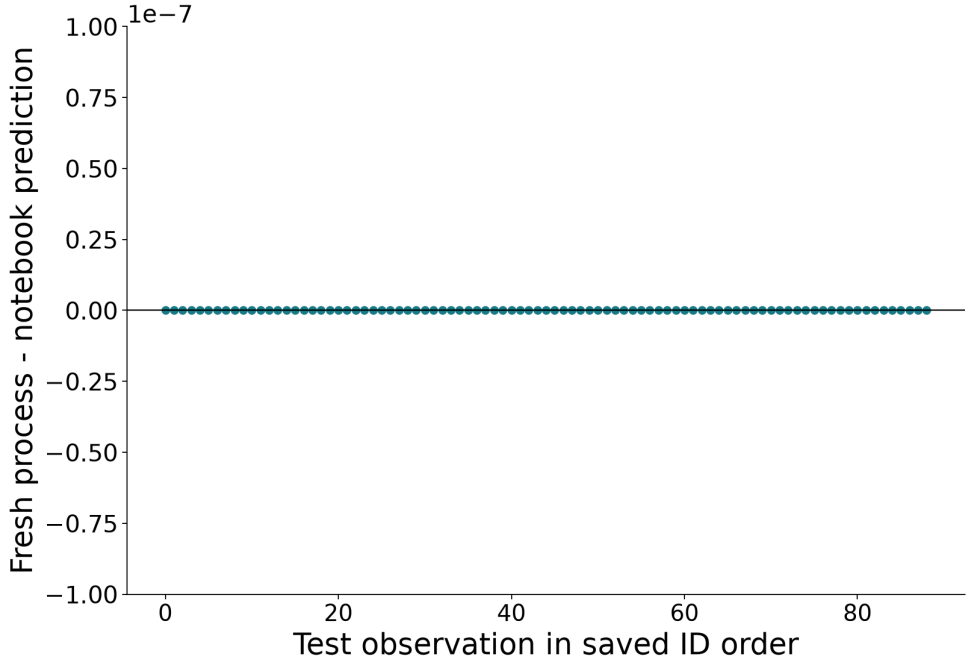


Figure 3: Actual fresh-process prediction differences after exact ID alignment. The vertical scale is explicit. Replay difference and residual against observed target answer different questions.

8 Test deliberate failures on copies

Corrupt only temporary copies of the data. Inject a duplicate ID, a non-finite value, or a reordered header. Catch the expected `ValueError` and record its message. An accepted malformed copy must trigger a failed assertion. Recheck the original data hash afterward.

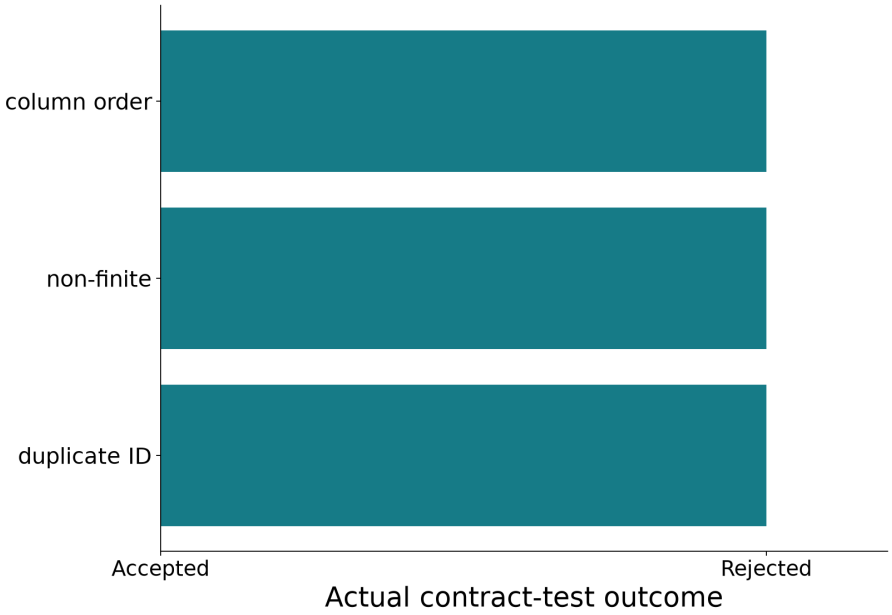


Figure 4: Actual three-case contract rejection test. These outcomes test an interface, not clinical safety or predictive accuracy.

9 Eight core exercises and two optional challenges

Each task requires a prediction before running, an actual observation, an explanation and a stopping check. Alternate driver and reviewer in pairs. Use H1 for Tasks 1–2, guided H2 for Tasks 3–5, and H3 for remaining work, review and export. These tasks share the 120 minutes of practice; they are not extra hours after the walkthrough.

1. **Row-order trap:** reverse a 20-ID toy dataset, reuse seeded positional indices, then recover frozen membership by ID. Explain set equality versus sequence equality.
2. **Preprocessing scope:** assert the fitted scaler mean equals training means. Contrast all-row means descriptively without fitting a leaky competing model or claiming a particular score change.
3. **Selection rule:** recompute the validation-MAE winner and declared tie order. Never use final-test results to choose it.
4. **Contract failures:** reject duplicate IDs, NaN and reordered columns on temporary copies. Confirm the original CSV unchanged.
5. **Fresh process:** execute the reviewed runner in another temporary working directory. Compare saved ID sequences and predictions under the explicit tolerance.
6. **Controlled extension:** fit alpha 200 on the same training rows and report validation only. Because final-test results have been seen, label this exploratory; a new selection claim needs a fresh evaluation protocol. Preserve the original winner.
7. **Local explanation:** add 0.1 training SD to one declared feature and verify the fitted-function prediction change. Record the candidate and exclude causal or clinical claims.

8. **Handover audit:** verify every payload digest and identify one actual failure row. Write claim, evidence, limitation and next evaluation so a partner can locate the supporting files.

Optional A changes a copied CSV and confirms manifest rejection before training. Optional B explicitly restores reordered numeric columns by unique names and verifies exact recovery of the canonical matrix. Both preserve the reference package and neither is required for core completion.

9.1 Student responses and evidence export

The 19 core fields comprise eight predictions, eight explanations, controlled change, limitation and AI-use/checking disclosure (or "none"). Published responses and outputs start blank. Missing fields produce status `draft`; nonempty fields mean `ready for peer discussion`, not a correctness grade. Rerun only the final export after editing responses; initialization resets them.

Download both the handover ZIP and exercise ZIP before the free runtime ends. After code review, the extracted runner can be executed in a compatible environment using the explicit root and output paths shown in the notebook. No paid AI, GPU, API key or external dataset download is needed. AI may help explain code or suggest hypotheses, but it must not fabricate execution evidence or measured values.

10 Instructor execution and limits

On 19 October 2026 both notebooks were fully executed in fresh local CPU Jupyter kernels. Each invoked independent replay interpreters. All eight core tasks, both optional challenges, draft/completed export branches, 11 plot exports, numerical checks and ZIP integrity were checked. This is not a claim that the Google Colab browser itself was used for execution. Actual versions are recorded in the evidence JSON; a local software check does not establish clinical, causal or external deployment validity.

Reference CSV SHA-256:

`a589d3035403d59869ec18c1007e379a396c9cc53b562cb979be676f2b85c4d4`

11 Summary and examination bridge

Freeze identities, choices, transformations and environment. Reject malformed interfaces before fitting. Compare aligned prediction vectors in a new process, then state exactly what agreement supports. Keep predictive error, replay error, file integrity and scientific validity separate. For the written and oral midterm, be ready to explain one choice, one failure and one change that would invalidate the claim.

12 Primary-source reading

scikit-learn: raw dataset API

scikit-learn: preprocessing pitfalls

scikit-learn: persistence and security

NumPy: random-stream compatibility